

API Monitoring 專案 Wiki

API Monitoring — Hybrid Storage · Project Wiki

一句話定位：一個具備 Cache-aside 快取策略、混沌工程異常模擬、以及完整可觀測性 (Observability) 的後端監控示範系統。

Table of Contents

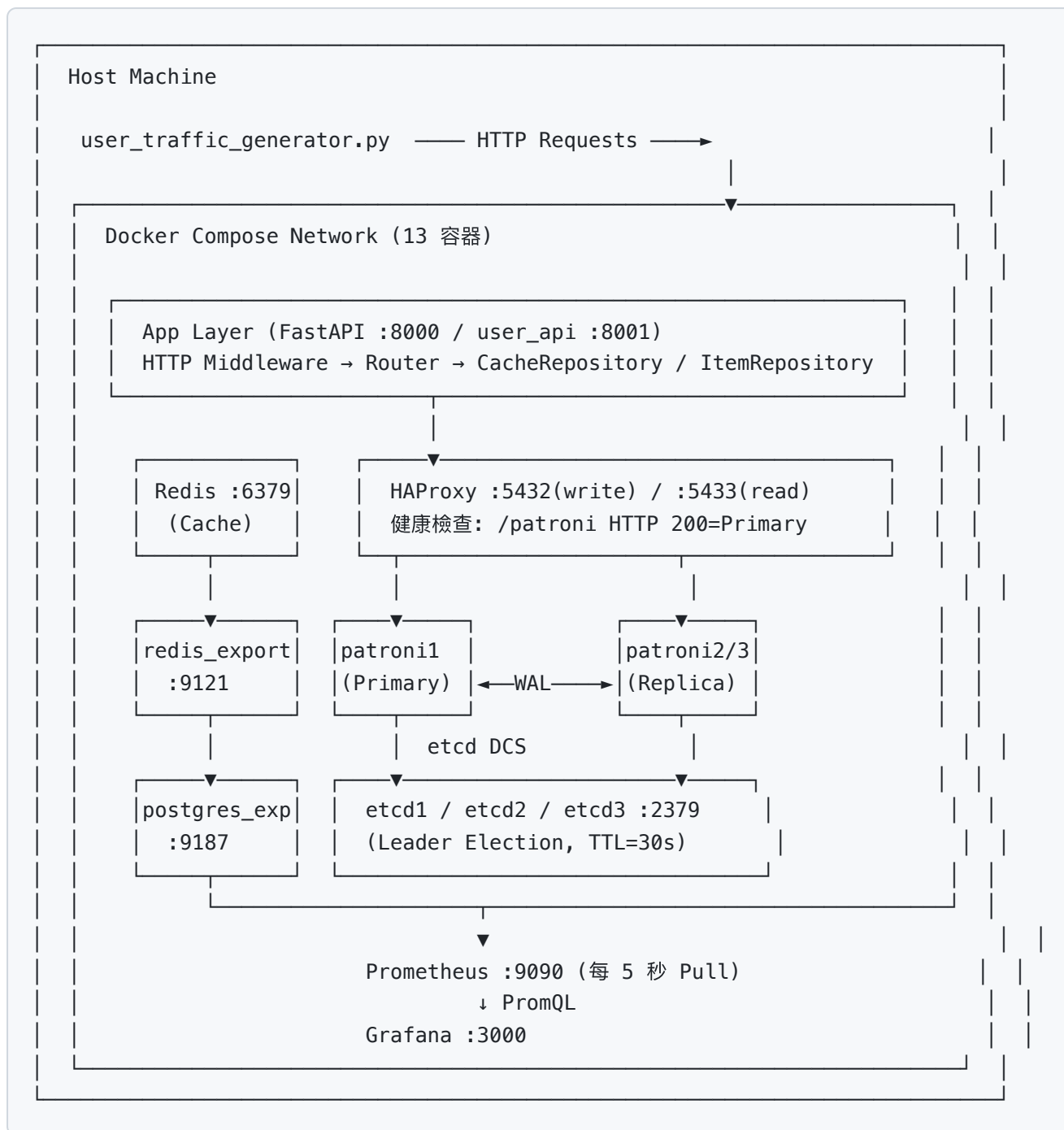
1. [Tech Stack & Services](#)
 2. [System Architecture](#)
 3. [Repository Structure](#)
 4. [Component Deep-Dive](#)
 - [4.1 config.py — 設定層](#)
 - [4.2 metrics.py — 指標層](#)
 - [4.3 CacheRepository — Redis 封裝](#)
 - [4.4 ItemRepository — PostgreSQL 封裝](#)
 - [4.5 routers/items.py — 業務路由](#)
 - [4.6 routers/anomaly.py — 混沌路由](#)
 - [4.7 main.py — App Factory](#)
 5. [Data Flows](#)
 - [5.1 Cache-aside 讀取流程](#)
 - [5.2 Metrics 採集流程](#)
 6. [API Reference](#)
 7. [Prometheus Metrics Reference](#)
 8. [Chaos Engineering Scenarios](#)
 9. [OOP Design Decisions](#)
 10. [How to Run](#)
-

1. Tech Stack & Services

類別	技術	版本	Port
HTTP Framework	FastAPI + Uvicorn	0.103.1 / 0.23.2	8000
User Service	user_api.py (FastAPI + Uvicorn)	host 直接執行	8001
Relational DB (HA)	PostgreSQL 15–bookworm via Patroni 4.1.3	3 nodes	HAProxy :5432
HA Proxy	HAProxy 2.8	write:5432 / read:5433 / stats:7001	5432/5433/7001
DCS	etcd 3.5.16	3 nodes (Raft Consensus), :2379/:2380	2379/2380
Cache (HA)	Redis 7–alpine — Sentinel HA	1 Master + 2 Replicas + 3 Sentinels	6379/6380/6381/26379
Metrics Scraper	Prometheus	latest	9090
Visualization	Grafana	13.0.1	3000
Redis Exporter	oliver006/redis_exporter	latest	9121
PostgreSQL Exporter	prometheuscommunity/postgres-exporter	latest	9187
Container Runtime	Docker Compose	3.8	—

共 18 個 Docker 容器：etcd1/2/3, patroni1/2/3, haproxy, api, redis-master, redis-replica1, redis-replica2, sentinel1/2/3, prometheus, grafana, redis_exporter, postgres_exporter

2. System Architecture



四條資料流：

- Main App Business Flow：** `user_traffic_generator` → FastAPI (8000) → Redis / HAProxy(5432) → Patroni Primary
- User API Business Flow：** `user_traffic_generator` → `user_api.py` (8001) → Redis / HAProxy(5432) → Patroni Primary
- Metrics Scrape Flow：** Prometheus 主動 Pull → FastAPI (8000) `/metrics` + `user_api` (8001) `/metrics` + 兩個 Exporter
- Visualization Flow：** Grafana 對 Prometheus 發 PromQL → 畫圖
- HA Failover Flow：** Patroni 透過 etcd DCS 自動選主 (RTO=5s) → HAProxy 自動重新路由

3. Repository Structure

```

api_monitoring/
├── docker-compose.yml      ← 定義並啟動全部 13 個容器 (含 HA 叢集)
├── user_api.py             ← 第二個 FastAPI 服務 (host 直接執行, port 8001)
                             模擬使用者行為: 搜尋/查詢/新增客戶
                             含 chaos 端點: /chaos/log-
├── storm/start|stop|status
                             暴露 user_api_* 指標供 Prometheus 採集
├── user_traffic_generator.py ← 雙服務流量產生器 + 混沌控制
                             同時打 user_api (8001) + main app (8000)
                             六階段互動式示範: --interactive
                             Grafana 所有面板全部有資料
├── logger_manager.py      ← 通用 logging 工具 (供 user_api.py 使用)
                             setup_logging(name) + @log_decorator
├── traffic_generator.py    ← (舊版, 已由 user_traffic_generator.py 取代)
├── chaos_scenario.py       ← (舊版, 已由 user_traffic_generator.py --
                             interactive 取代)
├── prometheus/
│   └── prometheus.yml      ← Prometheus scrape 任務設定
                             含 4 個 job: fastapi-app / redis / postgres /
├── user-api
├── patroni/
│   ├── patroni1.yml        ← Patroni 叢集設定
│   ├── patroni2.yml        ← patroni1 設定 (scope: pg-ha-cluster)
│   ├── patroni3.yml        ← patroni2 設定
│   └── patroni3.yml        ← patroni3 設定
├── haproxy/
│   └── haproxy.cfg         ← HAProxy 路由設定
                             pg_write frontend :5432 → Primary (/patroni
HTTP 200)
                             pg_read frontend :5433 → Replicas (/patroni
HTTP 503)
├── app/                   ← FastAPI 主 app 根目錄 (打包進 Docker image)
│   ├── Dockerfile
│   ├── requirements.txt
│   ├── config.py           ← Settings (frozen dataclass)
│   ├── metrics.py          ← AppMetrics (Singleton, 所有 Prometheus 指標)
│   ├── main.py             ← App Factory (組裝入口)
│   ├── repositories/
│   │   ├── cache_repo.py   ← 資料存取層 DAL
│   │   │                   ← CacheRepository (封裝 Redis)
│   │   └── item_repo.py     ← ItemRepository (封裝 PostgreSQL)
│   └── routers/
│       └── items.py         ← HTTP 路由層
                             ← /api/data, /api/write

```

```

├── anomaly.py          ← /api/anomaly/* (10 個 chaos 端點)
├── document/
│   ├── dir_structure.txt      ← 專案目錄說明
│   ├── system_architecture.mmd ← 系統架構圖 (Mermaid 格式) 含 HA 叢集
│   ├── grafana_panel_labels_guide.md ← Grafana 面板 legend 標籤對照表 (19 panels)
│   ├── project_wiki.md       ← 本文件
│   └── usage.txt             ← 啟動步驟

```

4. Component Deep-Dive

4.1 config.py — 設定層

設計模式：Immutable Value Object (frozen dataclass) + Factory Method

```

@dataclass(frozen=True)
class Settings:
    redis_sentinel_hosts: str # Sentinel 連線清單
    redis_master_name: str    # Sentinel 監控的 master 名稱
    database_url: str
    cache_ttl: int            # Redis TTL, 預設 10 秒

    @classmethod
    def from_env(cls) -> "Settings":
        ...

settings = Settings.from_env() # 模組層級單例

```

關鍵設計：– `frozen=True`：設定物件在執行期間不可被任何程式碼修改，防止意外的狀態變更 – `from_env()` Factory：統一從環境變數讀取，容易在測試時替換為固定值，不依賴系統環境 – `CACHE_TTL` 外部化：快取過期時間可透過 `docker-compose.yml` 的 `environment` 欄位調整，不需改程式碼

環境變數對應：

環境變數	預設值	說明
<code>REDIS_SENTINEL_HOSTS</code>	<code>sentinel1:26379,sentinel2:26379,sentinel3:26379</code>	Sentinel 節點清單
<code>REDIS_MASTER_NAME</code>	<code>mymaster</code>	Sentinel 監控的 master 名稱
<code>DATABASE_URL</code>	<code>postgresql://appuser:...</code>	PostgreSQL 連線字串
<code>CACHE_TTL</code>	<code>10</code>	Redis key 存活秒數

4.2 metrics.py — 指標層

設計模式：Singleton (`__new__` 覆寫)

```

class AppMetrics:
    _instance: "AppMetrics | None" = None

    def __new__(cls) -> "AppMetrics":
        if cls._instance is None:
            cls._instance = super().__new__(cls)
            cls._instance._register_metrics()
        return cls._instance

```

為何用 Singleton： `prometheus_client` 的限制是同名指標在同一個 Python 程序中只能被註冊一次。不管在哪個模組執行 `AppMetrics()`，都會回傳同一個實例，確保指標不會重複建立。

定義的所有指標：

指標名稱	類型	Labels	用途
<code>http_requests_total</code>	Counter	<code>method</code> , <code>endpoint</code> , <code>http_status</code>	API 請求計數，可算錯誤率
<code>http_request_duration_seconds</code>	Histogram	<code>method</code> , <code>endpoint</code>	回應時間分布，用於計算 P99
<code>redis_cache_hits_total</code>	Counter	—	Redis 命中次數
<code>redis_cache_misses_total</code>	Counter	—	Redis 未命中次數（退回 DB）
<code>redis_errors_total</code>	Counter	—	Redis 連線或操作錯誤次數
<code>db_errors_total</code>	Counter	<code>operation</code>	DB 操作失敗計數
<code>db_query_duration_seconds</code>	Histogram	<code>query_type</code>	DB 查詢耗時分布

Counter vs Histogram 的差別： – **Counter**：只增不減的累計數字，用 `rate()` 換算成每秒速率 – **Histogram**：把觀測值分配到多個 bucket，用 `histogram_quantile()` 計算百分位數（P50、P99）

4.3 `CacheRepository` — Redis Sentinel 封裝

職責： 所有 Redis Sentinel 操作的唯一入口，包含主節點自動發現、指標追蹤與錯誤處理。

```

CacheRepository
|
|— __init__(sentinel_hosts, master_name) ← Sentinel 連線初始化
|— ping() ← 啟動健康檢查
|— get(key) → Optional[str] ← 讀快取 (自動更新 hit/miss 計數)
|— setex(key, ttl, value) ← 寫快取 (失敗時靜默降級)
|— flush() ← 清空 Redis DB (混沌用)
|— simulate_connection_failure() ← 連接死節點 (混沌用)

```

Sentinel 初始化邏輯：

```

from redis.sentinel import Sentinel

def __init__(self, sentinel_hosts: str, master_name: str) -> None:
    hosts = [(h.split(":")[0], int(h.split(":")[1])) for h in
              sentinel_hosts.split(",")]
    _sentinel = Sentinel(hosts, socket_timeout=0.5, socket_connect_timeout=0.5)
    self._client = _sentinel.master_for(master_name, decode_responses=True)

```

Failover 行為： – Sentinel 偵測 master down (`down-after-milliseconds=5000`) – quorum=2 投票選出新 master (`failover-timeout=10000`) – redis-py Sentinel client 在下次呼叫 `master_for()` 時自動指向新主節點

錯誤降級策略 (Graceful Degradation)： – `get()` 發生錯誤 → 回傳 `None` (等同 Cache Miss) + `redis_errors` +1，主流程繼續打 DB – `setex()` 發生錯誤 → 靜默記錄 log，不拋出，不影響 HTTP 回應 – `flush()` 發生錯誤 → 拋出，由 Router 決定回傳 503

這個設計讓 Redis 掛掉不會導致 API 整體失敗，只是效能退化 (全部請求都打到 DB)。

4.4 ItemRepository — PostgreSQL 封裝

職責： 所有 PostgreSQL 操作的唯一入口，包含連線池管理、Schema 初始化、CRUD 與混沌操作。

```

ItemRepository
|
|— initialize_schema() ← 建表 + 塞 100 筆 seed data (冪等)
|— get_random_item() → dict ← SELECT RANDOM() LIMIT 1
|— insert_item(name, value) → float ← INSERT，回傳耗時
|— execute_slow_query() → float ← SELECT pg_sleep(3) (混沌用)
|— dispose() ← 釋放連線池

```

連線池設定 (SQLAlchemy)：

```

create_engine(
    database_url,
    pool_pre_ping=True,    # 取連線前先 PING，自動重連 (避免 idle connection 斷掉)
    pool_size=5,           # 最多維持 5 條持久連線
)

```

Schema :

```

CREATE TABLE IF NOT EXISTS items (
    id          SERIAL PRIMARY KEY,
    name        VARCHAR(100) NOT NULL,
    value       FLOAT        NOT NULL,
    created_at  TIMESTAMP    DEFAULT NOW()
);

```

`initialize_schema()` 是冪等的 (Idempotent) : `IF NOT EXISTS` 確保重複呼叫不會報錯，`COUNT(*)` 判斷只在空表時才塞資料。

4.5 routers/items.py — 業務路由

前綴： `/api`

依賴注入方式：從 `request.app.state` 取得 Repository 實例 (由 `lifespan` 在啟動時掛上)。

```

cache_repo = request.app.state.cache_repo
item_repo  = request.app.state.item_repo

```

這讓 Router 完全不依賴全域變數，Repository 的實作可以在不改 Router 程式碼的情況下被替換 (例如測試時換成 Mock)。

GET /api/data 完整邏輯：

```

1. cache_repo.get("data:latest")
   |
   |— Not None (Hit) → return {"source": "cache (Redis)", ...}
   |
   |— None (Miss)
       ↓
       item_repo.get_random_item()
           ↓
           cache_repo.setex("data:latest", TTL, result)
               ↓
               return {"source": "database (PostgreSQL)", ...}

```


4.6 routers/anomaly.py — 混沌路由

前綴： `/api/anomaly`

每個端點對應一種真實世界的故障場景：

端點	模擬場景	實作機制
<code>/lag</code>	整體系統變慢	<code>time.sleep(2.5)</code>
<code>/error</code>	應用程式拋出例外	<code>Response(status_code=500)</code>
<code>/db-overload</code>	資料庫慢查詢、index 遺失	<code>SELECT pg_sleep(3)</code>
<code>/cache-flush</code>	Cache 熱重啟、Stampede	<code>redis.flushdb()</code>
<code>/redis-down</code>	Redis 節點失聯	連接 <code>non-existent-redis:6379</code>

4.7 main.py — App Factory

職責： 唯一的組裝點，不包含任何業務邏輯。

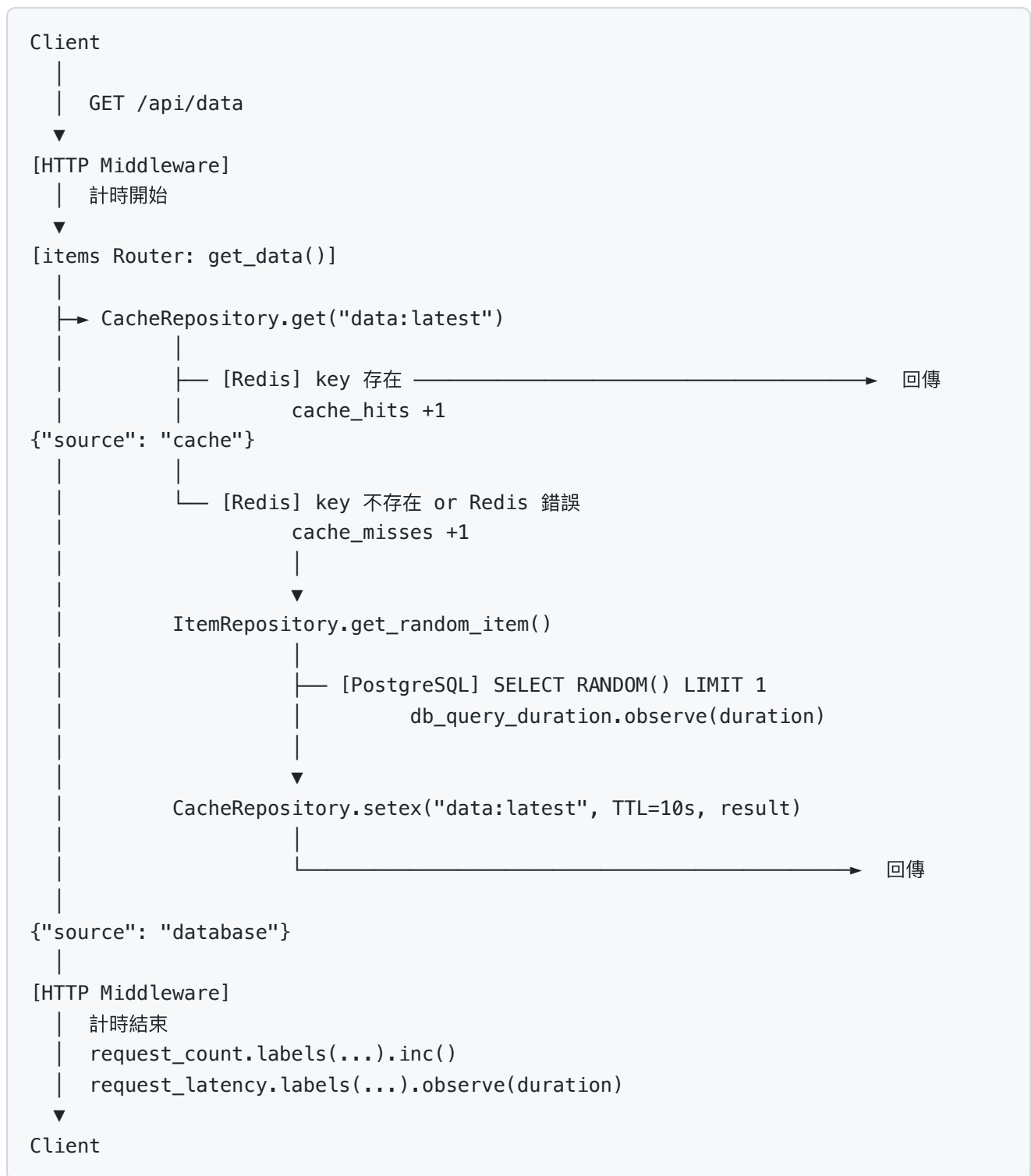
```
lifespan (啟動)
├─ CacheRepository(settings.redis_sentinel_hosts, settings.redis_master_name)
→ app.state.cache_repo
├─ ItemRepository(settings.database_url) → app.state.item_repo
├─ initialize_schema()

FastAPI app
├─ Middleware: monitor_requests (計時 + 寫指標)
├─ Mount: /metrics → Prometheus ASGI app
├─ include_router: items_router (/api/*)
├─ include_router: anomaly_router (/api/anomaly/*)

lifespan (關閉)
├─ item_repo.dispose()
```

5. Data Flows

5.1 Cache-aside 讀取流程



5.2 Metrics 採集流程

Prometheus (每 5 秒執行一次)

- └─ GET http://api:8000/metrics
 - http_requests_total
 - http_request_duration_seconds
 - redis_cache_hits_total
 - redis_cache_misses_total
 - redis_errors_total
 - db_errors_total
 - db_query_duration_seconds
- └─ GET http://redis_exporter:9121/metrics
 - redis_connected_clients
 - redis_memory_used_bytes
 - redis_commands_processed_total
 - ... (50+ Redis 底層指標)
- └─ GET http://postgres_exporter:9187/metrics
 - pg_stat_activity_count
 - pg_stat_database_tup_fetched
 - pg_locks_count
 - ... (100+ PostgreSQL 底層指標)

Grafana

- └─ PromQL 查詢 → Prometheus → 渲染 Dashboard

6. API Reference

業務端點

Method	Path	說明	回應範例
GET	/	健康檢查	<code>{"message": "API is running normally."}</code>
GET	/api/data	Cache-aside 讀取	<code>{"source": "cache (Redis)", "data": "..."} </code>
GET	/api/write	寫入隨機資料	<code>{"status": "written", "name": "item_1234", "value": 567.89}</code>
GET	/metrics	Prometheus 指標端點	純文字格式 (Prometheus exposition format)

混沌端點

Method	Path	預期行為	HTTP Status
GET	/api/anomaly/lag	延遲 2.5 秒後回應	200
GET	/api/anomaly/error	立即回傳錯誤	500
GET	/api/anomaly/db-overload	等待 3 秒 (DB 慢查詢) 後回應	200
GET	/api/anomaly/cache-flush	清空 Redis，立即回應	200
GET	/api/anomaly/redis-down	觸發 Redis 連線失敗	503

7. Prometheus Metrics Reference

關鍵 PromQL 查詢

API 錯誤率 (5xx)

```
sum(rate(http_requests_total{http_status=~"5.."}[1m]))
/
sum(rate(http_requests_total[1m]))
```

API P99 回應時間

```
histogram_quantile(0.99,
  sum(rate(http_request_duration_seconds_bucket[1m])) by (le, endpoint)
)
```

Redis Cache 命中率

```
rate(redis_cache_hits_total[1m])
/
(rate(redis_cache_hits_total[1m]) + rate(redis_cache_misses_total[1m]))
```

Redis 錯誤速率

```
rate(redis_errors_total[1m])
```

DB 慢查詢 P99

```
histogram_quantile(0.99,
  sum(rate(db_query_duration_seconds_bucket[1m])) by (le, query_type)
)
```

各 Status Code 請求速率

```
sum by (http_status) (rate(http_requests_total[1m]))
```

8. Chaos Engineering Scenarios

場景一：Cache Stampede（快取雪崩）

觸發： `GET /api/anomaly/cache-flush`

原理：1. Redis 快取被清空，所有 `data:latest` key 消失 2. 下一批 `/api/data` 請求全部 Cache Miss 3. 所有請求同時打到 PostgreSQL（流量尖峰） 4. 10 秒後 TTL 重建，Hit Rate 緩慢回升

觀察指標：– `redis_cache_misses_total` 暴增 – `db_query_duration_seconds` P99 短暫上升（DB 壓力增加） – Hit Rate 先降至 0% 後緩慢回升

場景二：Database Slow Query（DB 慢查詢）

觸發： `GET /api/anomaly/db-overload`

原理：– 執行 `SELECT pg_sleep(3)`，強迫 PostgreSQL 佔用連線 3 秒 – 模擬 Missing Index、Lock Contention、大量資料掃描等真實場景

觀察指標：– `db_query_duration_seconds{query_type="slow_query"}` P99 飆升至 ~3s – `http_request_duration_seconds` 整體 P99 也跟著升高

場景三：Redis Connection Failure（快取斷線）

觸發：GET `/api/anomaly/redis-down`

原理：– `CacheRepository.simulate_connection_failure()` 嘗試連接不存在的 `non-existent-redis:6379` – `socket_connect_timeout=1` 確保 1 秒內就會拋出例外 – `redis_errors_total` 計數增加 – 實際的 `/api/data` 流量不受影響（會從正常的 Redis 讀取），這個端點只是增加錯誤計數

觀察指標：– `redis_errors_total` 持續累加

場景四：HTTP 500 錯誤（應用程式崩潰）

觸發：GET `/api/anomaly/error`

觀察指標：– `http_requests_total{http_status="500"}` 增加 – Error Rate (5xx %) 上升

9. OOP Design Decisions

為什麼使用分層架構？



好處：– 單一職責：每個類別只有一個改變的理由 – 可測試性：Router 可以用 Mock Repository 進行單元測試 – 可替換性：日後把 Redis 換成 Memcached，只需改 `CacheRepository`，Router 零改動

為什麼 Repository 自己負責更新 Metrics？

Router 呼叫 `cache_repo.get(key)` 時不需要知道 `redis_cache_hits_total` 這個指標的存在，指標邏輯完全封裝在 Repository 內部。這遵循了 Tell, Don't Ask 原則。

為什麼用 `app.state` 做依賴注入而非全域變數？

```
# ❌ 舊做法：全域變數，測試困難，有競態條件風險
redis_conn = None

# ✅ 新做法：透過 app.state 注入，生命週期明確
app.state.cache_repo = CacheRepository(...)
```

`app.state` 的生命週期與 FastAPI app 相同，由 `lifespan` 統一管理，避免「物件已建立但連線未初始化」的問題。

10. How to Run

啟動全部服務

```
cd api_monitoring
docker-compose up -d --build
```

驗證各服務

```
# API 健康檢查
curl http://localhost:8000/

# Prometheus 指標 (確認埋點成功)
curl http://localhost:8000/metrics | grep http_requests

# Prometheus Web UI
open http://localhost:9090

# Grafana Dashboard (帳密: admin / admin)
open http://localhost:3000
```

產生流量

```
python traffic_generator.py
```

手動觸發異常場景

```
# Cache Stampede (觀察 Cache Hit Rate 驟降)
curl http://localhost:8000/api/anomaly/cache-flush

# DB 慢查詢 (觀察 DB P99 飆升至 3s)
curl http://localhost:8000/api/anomaly/db-overload

# Redis 斷線 (觀察 redis_errors_total 累加)
curl http://localhost:8000/api/anomaly/redis-down
```

停止服務

```
# 停止但保留容器 (資料不會消失)
docker-compose stop

# 停止並刪除容器 (資料會消失)
docker-compose down
```

查看 API 容器 Log

```
docker-compose logs -f api
```